

Reviewer Pack — Minimal Local Indeterminacy in Category Theory and Circuit M...

Entry 09b9468bd0ed · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 15:18:13 UTC

Minimal Local Indeterminacy in Category Theory and Circuit Monotone Width

Entry ID: 09b9468bd0ed **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 15:18:13 UTC

1. Conjecture

Field A (mathematical branch): Category Theory (Monoidal Categories) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For every Boolean circuit C with n inputs, the minimal local indeterminacy $\text{lind}(C)$ of its associated monoidal category K_C is linearly correlated with its circuit monotone width $w_{\text{mon}}(C)$, such that $\text{lind}(K_C) = \Theta(w_{\text{mon}}(C))$.

Rationale (proposer's reasoning):

Category theory provides a framework for studying algebraic structures and their relationships, which may uncover hidden structures in Boolean circuits. Monotone width captures the computational complexity of monotone functions, and we conjecture that it is related to the local indeterminacy within the category associated with a circuit.

Taxonomy category: category_theory_to_circuit_monotone_width (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 68525db2de0a4daf

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between the minimal local indeterminacy $\text{lind}(K_C)$ and the circuit monotone width $w_{\text{mon}}(C)$ across at least 1000 randomly generated Boolean circuits C with n inputs ($n \leq 40$) is ≥ 0.8 , AND the mean of $\text{lind}(K_C)$ is within a factor of 1.5 from $w_{\text{mon}}(C)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - lind(C) AND monoidal categories AND Boolean circuit complexity - circuit monotone width AND Category Theory AND K_C - minimal local indeterminacy AND $\theta(w_{\text{mon}}(C))$ AND Boolean circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.8s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
from itertools import combinations

def generate_circuit(n):
    circuit = []
    for i in range(n):
        gate_type = random.choice(['AND', 'OR'])
        inputs = sorted(random.sample(range(i), random.randint(1, i)))
        circuit.append((gate_type, inputs))
    return circuit

def monotone_width(circuit):
    n = len(circuit)
    if n == 0:
        return 0
    max_width = 0
    for i in range(n):
        width = 0
        for j in range(i + 1, n):
            if all(circuit[j][1][k] <= circuit[i][1][k] for k in range(len(circuit[i][1]))):
                width += 1
        max_width = max(max_width, width)
    return max_width

def minimal_local_indeterminacy(circuit):
    n = len(circuit)
    if n == 0:
        return 0
    indeterminacy = 0
    for i in range(n):
        for j in range(i + 1, n):
```

```

        if circuit[j][1].issubset(circuit[i][1]):
            indeterminacy += 1
    return indeterminacy

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for _ in range(30): # Ensure at least 30 instances per seed
        n = random.choice([5, 10, 15, 20, 30, 40])
        circuit = generate_circuit(n)
        w_mon = monotone_width(circuit)
        lind = minimal_local_indeterminacy(circuit)
        results.append((w_mon, lind))
    if not results:
        return {
            "metric_name": "correlation_coefficient",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }
    w_m = [w for w, _ in results]
    l_r = [l for _, l in results]
    n_max = max(n for _, _ in results)
    correlation_coefficient = sum((w - (sum(w_m) / len(w_m))) * (l - (sum(l_r) / len(l_r))) for w, l in zip(w_m, l_r)) / len(w_m)
    mean_lind = sum(l_r) / len(l_r)
    conjecture_holds = correlation_coefficient >= 0.8 and abs(mean_lind / (sum(w_m) / len(w_m)) - 0.5) < 0.1
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "not_enough_data"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='not_enough_data' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2655b95c.py", line 69, in <module>
    result = run_trial(seed)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2655b95c.py", line 52, in run_trial
    w_mon = monotone_width(circuit)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2655b95c.py", line 35, in monotone_width
    if all(circuit[j][1][k] <= circuit[i][1][k] for k in range(len(circuit[i][1]))):
        ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2655b95c.py", line 35, in <genexpr>
    if all(circuit[j][1][k] <= circuit[i][1][k] for k in range(len(circuit[i][1]))):
        ~~~~~
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its intended task of calculating the correlation coefficient and mean values for the conjecture. | next: Investigate the cause of the crash in the test code to ensure that future tests can be run successfully.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	23044
2	preregistration	ollama_remote	glm4:latest	0	0	16807
3	novelty	ollama_remote	glm4:latest	0	0	10006
4	novelty	ollama_remote	glm4:latest	0	0	15244
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20455
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14438
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	52605
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	39144
9	judge	ollama_remote	glm4:latest	0	0	8766

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 200508 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/09b9468bd0ed.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/09b9468bd0ed.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/09b9468bd0ed.tar.gz (if generated)